

**ANKARA UNIVERSITY
FACULTY OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING**



COM 491/492/461/462 PROJECT REPORT

Real Time Image Processing

Smart Security System

Hasan Baki İşeri

Mehmet İhsan Kırhan

21290161 - 20291266

Ayhan Aydın

04.01.2025

ABSTRACT

This document describes Real Time Image Processing Smart Security System, a real-time monitoring system for security cameras that is desktop-based, and includes motion detection, multi-class object detection, multi-object tracking, fire detection, evidence capture and metadata indexing, query-based searches and blacklist alerts. The system allows users to process both live webcam streams as well as offline video files, and provides output through a Graphical User Interface (GUI). By utilizing motion gating to detect humans, the amount of computing required for this process is reduced; however, the detection of vehicles, pets, and fires can occur independent of motion, by utilizing motion gating to object detection, the amount of computing required for this process is reduced. Object detection is conducted using a YOLOv8-based instance segmentation model that allows for the generation of masks on a per-object basis, thus enabling a more thorough analysis of the object's appearance. The application collects a set of frames representing an object's tracked identity and selects the 'best' frame from that set by calculating an image quality score, based on the following features: object size, image sharpness, centering, and aspect ratio plausibility. The 'best' frame is saved as an image with an overlay bounding box, and a record of the bounding box and the associated metadata is stored in a JSON-formatted metadata file. Human metadata includes upper and lower estimated clothing color; vehicle metadata consists of only a single color value that represents the vehicle's dominant object (for example, red for a car). The search module allows users to retrieve saved detections via time-range constraints and simple keyword queries in Turkish or English. The blacklist module enables users to create rules (such as items that would appear suspicious), and the module provides an alert pop-up whenever a newly-saved detection matches an existing rule in the blacklist.

Keywords: surveillance, real-time monitoring, instance segmentation, YOLOv8, SORT tracking, motion detection, fire detection, metadata indexing, search, blacklist alerts

TABLE OF CONTENTS

ABSTRACT.....	3
1. INTRODUCTION	6
1.1. Problem Definition	7
1.2. Scope.....	7
1.3. Design Goals.....	8
1.4. Contributions	9
1.5. Report Organization	9
2. BACKGROUND	10
2.1. Motion Detection	10
2.2. Object Detection and Instance Segmentation.....	10
2.3. Online Tracking (SORT).....	10
2.4. Attribute/Appearance Estimation (Color)	11
2.5. Fire Detection Heuristics	11
3. REQUIREMENTS AND SYSTEM ANALYSIS.....	13
3.1. Functional Requirements.....	13
3.2. Non-Functional Requirements.....	13
3.3. Project Artifacts and Data.....	13
4. SYSTEM DESIGN	14
4.1. High Level Architecture	14
4.2. Control Flow	14
4.3. Design for Evidence Quality	15
4.3. Metadata Design	15
5. IMPLEMENTATION DETAILS	16
5.1. Motion Detection Module.....	16
5.2. Human Detection with Segmentation + Tracking	16
5.3. Vehicle and Pet Detection	16
5.4. Fire Detection Module	16
5.5. Frame Skipping Strategy	17
5.6. Quality Score and Best-Frame Selection	17
5.7. Saving Evidence and Writing Metadata	18
5.8. Search Implementation	18
5.9. Blacklist Alerts.....	18
6. EXPERIMENTS AND RESULTS	19
6.1. Experimental Setup	19
6.2. Qualitative Observations	19
6.3. Search Demonstration Scenarios (Examples)	19
7. PERFORMANCE EVALUATION	20
7.1. Metrics	20
7.2. Methodology.....	20

7.3.	Expected Bottlenecks and Mitigations	20
7.4.	Discussion.....	21
8.	DISCUSSION, LIMITATIONS, AND FUTURE WORK.....	22
8.1.	Limitations.....	22
8.2.	Future Improvements	22
8.3.	Ethical and Privacy Notes.....	24
9.	CONCLUSION.....	25
	BIBLIOGRAPHY	26

1. INTRODUCTION

Security systems are widely used in residential, commercial and industrial applications. However, the effectiveness of these systems in crowded public environments is limited due to dependence on continuous recording and human operatives. In many cases security personnel must continuously view multiple video feeds for extended periods of time, which leads to fatigue, delayed response times and increased operational costs; and due to the nature of the changing scenes that characterize these environments, event detection and reliable event labelling is difficult.

This project aims to develop an intelligent, real-time security monitoring system for high-risk public areas with minimum dependence on continuous human monitoring. The system will automatically detect and record incidents similar to those that would have been identified by a skilled operative, thereby allowing the operator to focus on tasks that require greater expertise or skill levels, while also significantly reducing the amount of work placed on the operator by automating much of the video monitoring process.

Real Time Image Processing Smart Security System utilizes a single desktop application for motion detection (i), detecting and tracking vehicle and pet entities (ii), identifying heuristic fire-like regions (iii), and auto-storing representative evidence frames for tracked identities (iv). The application also has a structured metadata system for event detection, which allows for faster, easier queries of archived evidence records (vi), as well as provides real-time alerts for matches to a defined blacklist (vii).

The proposed system will convert long-format video records into images by doing so, it transforms the security monitoring process from a human-centric, passive observation of video feeds to an automated and intelligence-enhanced task. The end result will be less continuous and subsequent effectiveness in monitoring environments that require rapid and reliable detection to ensure the protection of the public.

1.1. Problem Definition

Despite the ubiquity of video surveillance, many current security monitoring solutions have limitations which affect their ability to operate effectively in public settings. Current security monitoring solutions will need to operate in real-time, while handling large amounts of visual information, making it difficult to find meaningful events among non-meaningful activity.

There are a number of specific challenges that need to be addressed by the security monitoring solution:

1. Identify security-relevant objects and events in real-time, without requiring constant attention from a human operator.
2. Avoid the storage of redundant or low-value frames, which results in inefficient use of storage and makes subsequent review of the video archive unnecessarily complex.
3. Extract and assign structured, searchable attributes to detected objects and events, regardless of the complexity of the scene and visual clutter.
4. Enable quick retrieval of security-relevant events based on time, physical characteristics, or an associated object type from large volumes of video data.
5. Provide an easy-to-use interface for operators to quickly review and manage detected security-related events without the need for constant monitoring of live feeds.

1.2. Scope

In this project, we will implement a single-machine based, real-time intelligent Security Monitoring System using a video camera (webcam) or with pre-recorded video files in public areas. The major sections of this project scope are:

The real-time capture and processing of the video input

Detecting motion using a method called Background Subtraction to identify the areas, in a scene, that have changed.

Human, vehicle, and pet detection using an enacted deep-learning object detection algorithm (using the pretrained instance segmentation model).

Maintaining an online association of multiple objects throughout the sequence of frames (by tracking).

Detecting fire using HSV-based color thresholding and temporal flickering analysis techniques.

Storage of evidence frames of detected human and vehicle movements, in conjunction with the creation of a JSON file of indexed metadata that allows subsequent retrieval of that evidence.

A graphical user interface that allows for module (component enable/disable) control, time-based navigation, and search by executing queries against stored evidence data.

An ability to define blacklists and have real-time alerts generated for detected persons matching those blacklists.

1.3. Design Goals

- Responsive: To ensure a user-friendly experience that is fluid and enjoyable.
- Reliable: Evidence and associated media metadata should be saved without being changed.
- Retrievable: Searches should be pragmatic and should not require excessive user time or effort.
- Efficient: Inference calls requiring excessive data processing should be minimised through means such as frame skipping (due to motion gating).
- Quality of Evidence: A single representative frame should be recorded for each valid identity being tracked by the system.

1.4. Contributions

- Unified video surveillance pipeline solution (motion, segmentation, detection, indexing).
- Explicit quality score on per-track basis allowing a user to select the highest rated frame of a track.
- Detection of clothing/vehicle appearance attributes via extracted colour awareness from associated masks.
- Ability to search metadata stored in JSON format using time range filtering and parsed query options.
- Blacklist notifications are displayed and can be disabled to prevent the presentation of duplicate alerts.

1.5. Report Organization

Chapter 2 explains background theory. Chapter 3 discusses the system's functional and non-functional requirements and performs a systems analysis. Chapter 4 discusses the design of the system. Chapter 5 describes how the system functions. Chapter 6 presents the results of the experiments. Chapter 7 includes the performance evaluation information. Chapter 8 describes potential limitations and future improvements. Chapter 9 presents the concluding remarks.

2. BACKGROUND

2.1. Motion Detection

Often, the first step for identifying frames that can hold important activity through motion detection. Motion detection is performed through methods of background subtraction that model how a scene is static in nature and will indicate when an object changes within that scene. In the case of Real Time Image Processing Smart Security System, motion detection serves as the "gate" into human detection in that Real Time Image Processing Smart Security System's deep inference calls will not be conducted unless there is an actual movement in the frame. Surveillance scenes tend to stay very static, with minimal activity occurring.

Motion detection also encounters some challenges, such as shadows and lighting fluctuations. By incorporating a shadow-aware background subtractor and filtering through contour area, the chances of false alarms are reduced.

2.2. Object Detection and Instance Segmentation

Detecting objects provides bounding boxes and class labels. On top of detecting objects, Instance segmentation predicts a binary mask for each instance of an object that is detected. According to the study on using masks for appearance analysis, masks remove background pixels that would introduce noise in calculating color.

Real Time Image Processing Smart Security System relies on the YOLOv8 instance segmentation and filters based on the COCO classes:

- person (for humans),
- car/motorcycle/bus/truck (for vehicles) and
- cat/dog (for pets).

2.3. Online Tracking (SORT)

Tracking retains an object's unique identity in all frames. An example of a popular tracker is SORT (Simple Online Real-Time Tracking), which consists of:

- A Kalman filter to predict an object's state,
- A method of associating objects through Intersection over Union (IoU),
- And the use of the Hungarian algorithm to match objects.

For this project, tracking is necessary to:

- Avoid saving the same person/vehicle multiple times,
- Enable collection of multiple candidate images for each identified person or vehicle so they can be reviewed later when selecting "the best frame"

2.4. Attribute/Appearance Estimation (Color)

Color extraction from images presents several challenges, These challenges come from varying illumination, motion blur, camera colour response, and background pixels close to the object boundary.

In order to provide better colour identification when extracting colour from an image using Real Time Image Processing Smart Security System, the Background Pixel Removal tool creates segmentation masks to whiten the background pixels, samples the object regions and applies K-Means clustering and HSV-based colour naming.

To confirm the accuracy of every user's clothing colour extraction, the upper and lower bounding boxes are separated to identify the clothing colour of each user individually.

2.5. Fire Detection Heuristics

The implementation of fire detection is through lightweight heuristic methods:

- The HSV Thresholding method involves utilizing the Red, Orange and Yellow colour ranges (through the use of colour designation in HSV).

- Brightness constraints are placed to suppress brightly lit LED's.
- Temporal Flickering refers to the temporal mask filtering performed on recently captured frames to generate a mask of temporal flickering.
- Contour Filtering refers to filtering contours based on Source Area, Aspect Ratio and irregularity (Extent).

The Fire Detection Heuristic method does not replace the need for a Specialized or fully developed fire detection model, however, it can serve as an Early Warning System for specific environments.

3. REQUIREMENTS AND SYSTEM ANALYSIS

3.1. Functional Requirements

- FR1: Capture frames from webcam and from a selected video file.
- FR2: Display live video with overlays for detected objects/events.
- FR3: Detect motion and optionally use it to gate human detection.
- FR4: Detect humans, vehicles, and pets; detect fire regions.
- FR5: Track detected humans/vehicles/pets with persistent IDs.
- FR6: Save evidence images for humans and vehicles; save pet images.
- FR7: Create and maintain metadata indexes (`metadata.json`) for humans and vehicles.
- FR8: Support query-based search by time range and keywords.
- FR9: Provide blacklist management and alerts on matches.
- FR10: Log key events to a log file.

3.2. Non-Functional Requirements

- NFR1 (Usability): GUI must support non-technical users.
- NFR2 (Performance): Must remain responsive in real time.
- NFR3 (Robustness): Must handle missing camera/video errors gracefully.
- NFR4 (Maintainability): Modular code for detectors and utilities.
- NFR5 (Portability): Run on common desktop OS setups with Python.

3.3. Project Artifacts and Data

The system produces:

- `logs/detection.log` for event logging,
- `detected\_humans/` with evidence images and `metadata.json`,
- `detected\_cars/` with evidence images and `metadata.json`,
- `detected\_pets/` with evidence images,
- `blacklist.json` for blacklist rules.

4. SYSTEM DESIGN

4.1. High Level Architecture

Real Time Image Processing Smart Security System follows a layered architecture:

- Presentation Layer (GUI):
 - live camera display,
 - controls (toggles, detection hours, video load/pause),
 - search panel and results grid,
 - blacklist management and alerts,
 - logs/info text area.
- Processing Layer:
 - frame capture loop (threaded),
 - frame skipping scheduler,
 - detection pipeline orchestration,
 - drawing overlays for display.
- Vision Modules:
 - motion detector,
 - human detector (YOLOv8-seg + tracking),
 - vehicle detector (YOLOv8-seg + tracking),
 - pet detector (YOLOv8-seg + tracking),
 - fire detector (HSV + flicker).
- Storage and Indexing:
 - saving evidence frames,
 - metadata extraction,
 - JSON read/write utilities,
 - duplicate suppression and per-track best-frame selection.

4.2. Control Flow

1. Initialize GUI, detectors, and trackers.

2. Start capture from webcam (default) or selected video.
3. For each frame:
 - update display frame,
 - periodically (every Nth frame) run detections,
 - maintain track IDs and candidate frames,
 - on completion or track loss, save best evidence,
 - update metadata JSON,
 - check blacklist and trigger alerts if matched.
4. Support pause/resume for video playback.
5. Support search over metadata and display results.

4.3. Design for Evidence Quality

Saving every detection frame produces redundant data and may store blurred or partial views. Real Time Image Processing Smart Security System uses:

- a candidate buffer per track ID,
- a quality score function,
- best-frame selection.

This produces fewer but higher-quality evidence images and reduces storage useage.

4.3. Metadata Design

Two separate metadata files are used:

- humans: includes upper/lower clothing fields,
- vehicles: includes colors and object_type.

Metadata includes timestamp and hour to support time-range filtering.

5. IMPLEMENTATION DETAILS

5.1. Motion Detection Module

Motion detection is implemented via background subtraction (MOG2) and thresholding. Contours are extracted and motion is considered true if a contour exceeds a minimum area. This module returns a boolean and a mask.

Rationale: inexpensive detection of “activity” frames for gating.

5.2. Human Detection with Segmentation + Tracking

Humans are detected by running YOLOv8 segmentation restricted to class “person.” The output provides bounding boxes and instance masks. Detections are passed to SORT to obtain a stable `track\_id`. The returned tuple includes bounding box coordinates and the mask.

Rationale: segmentation masks improve later metadata extraction (clothing colors).

5.3. Vehicle and Pet Detection

Vehicle and pet detection follow the same pattern but use different class filters:

- Vehicles: car/motorcycle/bus/truck.
- Pets: cat/dog.

Tracking provides stable IDs for duplicate suppression and for consistent evidence naming.

5.4. Fire Detection Module

Fire detection uses HSV masks for fire-like colors plus:

- morphological operations to reduce noise,

- brightness constraints to reduce false detections from lighting,
- flicker detection across recent frames,
- contour filtering based on size and shape irregularity.

Bounding boxes are returned for overlay and logging

5.5. Frame Skipping Strategy

Heavy modules (YOLO inference) are not run on every frame. The application processes detections when a motion is detected to reduce computational load while still updating the display smoothly on every frame.

5.6. Quality Score and Best-Frame Selection

For each track ID, the system stores up to a limited number of candidate frames. Each candidate stores:

- a copy of the frame,
- the detection tuple and bounding box,
- timestamp,
- quality score.

Quality score combines:

- object size ratio in frame,
- Laplacian variance for sharpness,
- distance from center,
- plausible aspect ratio.

When enough candidates are collected or the track is lost, the best candidate is saved.

5.7. Saving Evidence and Writing Metadata

Evidence images are saved as full frames with bounding boxes and labels. Metadata entries are appended to JSON:

- Humans: upper/lower clothing colors, bbox, timestamp, track_id, quality_score.
- Vehicles: dominant colors list, object type, bbox, timestamp, track_id, quality_score.
- Pets: saved without JSON metadata (simplified design).

5.8. Search Implementation

Search is performed by:

- parsing the query for color and object keywords (Turkish/English),
- filtering metadata entries by hour range,
- applying matching logic depending on whether the query implies “vehicle search” or not,
- sorting results by a confidence-like score and displaying thumbnails.

5.9. Blacklist Alerts

Blacklist entries are stored as JSON records. A new saved detection is checked against the blacklist:

- human: match against clothing colors,
- vehicle: match against colors/type,
- pet: match against type (cat/dog).

A cooldown mechanism prevents repeated alert spam for the same rule.

6. EXPERIMENTS AND RESULTS

6.1. Experimental Setup

Two usage scenarios were tested:

- Webcam mode: live monitoring in a typical indoor environment.
- Video mode: processing recorded videos to validate offline analysis and search.

Resolution was set to 640×480 for performance. Detection modules can be toggled in the GUI.

6.2. Qualitative Observations

- Best-frame selection improves saved evidence clarity compared to saving arbitrary frames.
- Clothing color estimation is generally plausible in stable lighting; errors can occur under shadows or strong reflections.
- Fire detection can highlight fire-like flickering regions; however, strong colored lights may still cause false positives in edge cases.

6.3. Search Demonstration Scenarios (Examples)

The search system supports queries such as:

- “blue shirt”
- “red bus”
- “black car”

and filters results by time range (hours).

7. PERFORMANCE EVALUATION

7.1. Metrics

- Display FPS: perceived smoothness of GUI updates.
- Effective detection rate: detection frequency given frame skipping.
- Latency: delay between a visible event and overlay/alert.
- CPU usage: average and peak.
- Memory usage: Python process working set.
- Disk growth: size increase due to saved evidence images and metadata.

7.2. Methodology

Because hardware details are intentionally left blank, we define a reproducible measurement procedure:

1. Run webcam mode for 10 minutes with all detectors enabled.
2. Record CPU and memory usage using the operating system performance tools.
3. Observe detection rate: if the camera runs near 30 FPS and detection runs every 2 frames, effective detection rate is approximately 15 detections/sec for enabled modules.
4. Measure disk usage difference of `detected\_`*` directories before and after the run.

7.3. Expected Bottlenecks and Mitigations

- Main bottleneck: YOLOv8 segmentation inference.
- Mitigations implemented:
 - frame skipping (run detection every N frames),
 - motion gating for human detection,
 - saving only best frames per track.

7.4. Discussion

The system balances detection richness (multiple modules) with responsiveness using frame skipping and gating. Best-frame selection reduces storage and I/O overhead while providing better evidence quality for later review

8. DISCUSSION, LIMITATIONS, AND FUTURE WORK

8.1. Limitations

- Color estimation sensitivity: lighting changes and reflections can reduce accuracy.
- Heuristic fire detection: may produce false positives/negatives compared to learning-based detectors.
- Search semantics: keyword-based parsing; does not fully understand complex language.
- Tracking drift: SORT may lose IDs under occlusion or rapid motion.

8.2. Future Improvements

Even though the proposed system exemplifies a working real, time smart surveillance prototype, it has several limitations, which were observed during experimental evaluations. The primary focus of future works will be on upgrading the reliability of the existing modules and the system's scalability.

Clothing Detection and Attribute Analysis Improvements

The current method of clothing color extraction is by instance segmentation masks in conjunction with K, Means clustering. This method, although computationally efficient, is also very sensitive to changes in illumination, occlusions, and background noise. As a next step, models based on human parsing such as LIP, CIHP, or SCHP can be used. These models offer fine, grained semantic segmentation of human body parts (e.g., upper clothing, lower clothing, shoes), which allows for more correct and sturdy clothing attribute extraction, regardless of the background noise. By employing these models, maybe the consistency of the clothing attributes stored would be greatly enhanced, and fewer colors would maybe falsely classified.

Fire Detection Module Enhancement

Experimental results reveal that the existing fire detection module, which is based on HSV color thresholding and simple temporal variation analysis, lacks sufficient robustness in complex lighting conditions and often generates false positives.

Subsequent system versions might substitute this method with a deep learning, based fire detection model trained on flame and smoke datasets. In addition, a hybrid pipeline can be used where classical image processing is only utilized as a candidate region generator, followed by a lightweight CNN, based fire verification stage to obtain high accuracy while maintaining real, time performance.

Distributed Alert and Notification System

At present, alerts are produced locally within the application interface. As a further development, the system can be connected to a distributed notification network that allows real, time alerts to be sent to external devices like mobile phones, tablets, or remote monitoring terminals. This may be implemented through network, based communication protocols or cloud, assisted messaging services, thus enabling security personnel to receive instant notifications no matter where they are.

Hybrid Camera-Embedded System Architecture

The entire detection and analysis pipeline runs on a single host machine in the current setup. To alleviate this heavy computational burden and make the system more scalable, the next phase of development could be a hybrid architecture combining conventional cameras with embedded edge devices. Inexpensive operations such as motion detection, frame sampling, or rudimentary object detection can be locally

performed on an embedded platform, whereas the main system can be allotted with the heavier operations. Such distributed processing will markedly lower the CPU load of the host machine and make multi, camera scenarios more feasible and efficient.

8.3. Ethical and Privacy Notes

This system can capture personal data. Any deployment must comply with privacy laws and obtain appropriate consent. Access control and secure storage are recommended in real-world usage.

9. CONCLUSION

Real Time Image Processing Smart Security System demonstrates an integrated real-time surveillance assistant that combines motion detection, instance segmentation-based object detection, object tracking, fire detection, evidence saving, metadata indexing, query-based search, and blacklist alerts within a single GUI application. The system successfully produces structured evidence archives for humans and vehicles and supports practical retrieval by time and appearance/type. Performance considerations are addressed through motion gating and frame skipping, and evidence quality is improved through best-frame selection. The project provides a foundation for further improvements toward higher robustness, richer search, and enhanced security features.

BIBLIOGRAPHY

- Al-Ghaili, A. M., Gunasekaran, S. S., Jamil, N., Alyasseri, Z. A. K., Al-Hada, N. M., Ibrahim, Z. B., Bakar, A. A., Kasim, H., Hosseini, E., Omar, R., Kasmani, R. M. and Razali, R. A. 2023. A Review on Role of Image Processing Techniques to Enhancing Security of IoT Applications. *IEEE Access*.
- Bewley, A., Ge, Z., Ott, L., Ramos, F. and Upcroft, B. 2016. Simple Online and Realtime Tracking. 2016 IEEE International Conference on Image Processing (ICIP).
- Kongurrsa, N., Chumuang, N. and Ketcham, M. 2017. Real-time Intrusion - Detecting and Alert System by Image Processing Techniques. 2017 10th International Conference on Ubi-media Computing and Workshops (Ubi-Media).
- Redmon, J., Divvala, S., Girshick, R. and Farhadi, A. 2016. You Only Look Once: Unified, Real-Time Object Detection. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Ur Rehman, A., Ullah, N., Shehzad, N. and Mohsan, S. A. H. 2021. Image Processing Based Detecting and Tracking for Security System. *International Journal of Engineering (IJE)*.
- Thakkar, H., Tambe, N., Thamke, S. and Gaidhane, V. K. 2020. Object Tracking by Detection using YOLO and SORT. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology (IJSRCSEIT)*.
- Wojke, N., Bewley, A. and Paulus, D. 2017. Simple Online and Realtime Tracking with a Deep Association Metric. 2017 IEEE International Conference on Image Processing (ICIP).
- Terven, J., Córdova-Esparza, D. and Romero-González, J. A. 2023. A Comprehensive Review of YOLO: From YOLOv1 to YOLOv8 and Beyond.

- Zivkovic, Z. 2004. *Improved Adaptive Gaussian Mixture Model for Background Subtraction. Proceedings of the 17th International Conference on Pattern Recognition (ICPR).*
- Töreyin, B. U., Dedeoğlu, Y., Güdükbay, U. and Çetin, A. E. 2006. *Computer Vision Based Method for Real-Time Fire and Flame Detection. Pattern Recognition Letters.*

